

Yo team, update on last night's rabbit hole lol
s(CASP) is goal directed, so with one broad query like
action(Actor, Action)
it only follows proof paths relevant to that action query instead of grounding every possible
state and action combination. The main idea is **candidate actions plus defeat
exceptions**.***
exceptions handle precedence, constraints handle numeric relationships without turning
them into weights. So weights combine numeric facts into an arbitrary score while using
constraints instead uses numeric facts directly as logical conditions.
Bro, the weights idea kind of defeats itself if you actually walk through it.
Assume weights are supposed to be the simpler solution.
We still have to write all the exact same game logic either way. We still need to know if a
barricade is urgent, if a lever is safe, if a zombie is close, if the player is in danger, if a
weapon is needed, etc. The weights cannot exist without all of that logic first.
Then after doing all that work, the weighted version adds another entire layer on top

Unity sends the current game state facts!

example:

```
lever_visible(player1, Lever).  
zombie_at_barricade(Zombie, Barricade).  
distance(player1, Lever, Distance).  
distance(Zombie, Lever, ZombieDistance).  
at_barricade(player1, Barricade).  
health(player1, Health).  
weapon_owned(player1, Weapon).
```

Notes:

#< means “must be less than”
#=< means “must be less than or equal to”
#> means “must be greater than”
#>= means “must be greater than or equal to”
#= means “must be equal to”
#\= means “must not be equal to”

example:

```
candidate_action(Actor, hunt_lever) :-  
    innocent(Actor),  
    levers_remaining.
```

```
candidate_action(Actor, pull_lever(Lever)) :-  
    innocent(Actor),  
    lever_available(Actor, Lever).
```

```
candidate_action(Actor, hold_barricade(Barricade)) :-  
    innocent(Actor),  
    urgent_barricade(Actor, Barricade).
```

```
candidate_action(Actor, move_to_barricade(Barricade)) :-  
    innocent(Actor),  
    urgent_barricade(Actor, Barricade),  
    not at_barricade(Actor, Barricade).
```

```
candidate_action(Actor, find_weapon) :-  
    innocent(Actor),  
    weapon_needed(Actor).
```

```
candidate_action(Actor, run_from(Entity)) :-  
    immediate_danger_from(Actor, Entity).
```

```
defeated(Actor, hunt_lever) :-  
    urgent_barricade(Actor, _).
```

```
defeated(Actor, pull_lever(_)) :-  
    immediate_danger(Actor).
```

```
defeated(Actor, hold_barricade(_)) :-  
    immediate_danger(Actor).
```

```
defeated(Actor, move_to_barricade(_)) :-  
    immediate_danger(Actor).
```

```
defeated(Actor, find_weapon) :-  
    urgent_barricade(Actor, _).
```

```
defeated(Actor, find_weapon) :-  
    immediate_danger(Actor).
```

```
action(Actor, Action) :-  
    candidate_action(Actor, Action),  
    not defeated(Actor, Action).
```

So hunt_lever can normally succeed, but the moment there is an urgent barricade, that specific action is defeated. We are not generating every action with every modifier and ranking all of them. We are pruning bad actions directly through the logic.

If two actions still survive, that means the model is missing a real distinction. Then we add a local check like safety, distance, danger, or reachability. For example, if two levers are valid, prefer the safe one meaning zombies are further from it. Then if both are safe, choose the closer one. That is not combinatorial explosion. It is just adding the actual game rule that separates two otherwise valid choices.

So the flow is basically:

- facts
- candidate actions
- defeat exceptions
- local tie checks
- final action

No weights, no Python ranking, and no need to evaluate some giant product of every possible combination. I truly think this is the way, thoughts?

CONSTRAINTS

% Unity provides the raw numeric facts.
% s(CASP) uses constraints to interpret what those numbers mean.

% An entity is an immediate danger if it is within 2 units.
immediate_danger_from(Actor, Entity) :-
 distance(Actor, Entity, Distance),
 Distance #=< 2.

% Actor is in immediate danger if ANY entity satisfies
% immediate_danger_from.
immediate_danger(Actor) :-
 immediate_danger_from(Actor, _Entity).

HEALTH

3 = full health
2 = has taken one hit
1 = one hit away from death
0 = dead

critical_health(Actor) :-
 health(Actor, Health),
 Health #=< 1.

low_health(Actor) :-
 health(Actor, Health),
 Health #=< 2.

LEVER DANGER

% A lever is threatened if the nearest zombie is within 5 units.

lever_threatened(Lever) :-
 nearest_zombie_distance_to_lever(Lever, Distance),
 Distance #=< 5.

% A lever is considered safe when the nearest zombie
% is farther than 5 units away.

lever_safe(Lever) :-
 nearest_zombie_distance_to_lever(Lever, Distance),
 Distance #> 5.

DISTANCE COMPARISON

% Target1 is closer to Actor than Target2.

closer(Actor, Target1, Target2) :-
 distance(Actor, Target1, Distance1),
 distance(Actor, Target2, Distance2),
 Distance1 #< Distance2.

LEVER COMPARISON

% Safe lever beats threatened lever.

better_lever(Actor, Lever1, Lever2) :-

```
    lever_available(Actor, Lever1),  
    lever_available(Actor, Lever2),  
    Lever1 \= Lever2,  
    lever_safe(Lever1),  
    lever_threatened(Lever2).
```

% If BOTH levers are safe, choose the closer one.

better_lever(Actor, Lever1, Lever2) :-

```
    lever_available(Actor, Lever1),  
    lever_available(Actor, Lever2),  
    Lever1 \= Lever2,  
    lever_safe(Lever1),  
    lever_safe(Lever2),  
    closer(Actor, Lever1, Lever2).
```

% If BOTH levers are threatened, choose the one whose
% nearest zombie is farther away.

better_lever(Actor, Lever1, Lever2) :-

```
    lever_available(Actor, Lever1),  
    lever_available(Actor, Lever2),  
    Lever1 \= Lever2,  
    lever_threatened(Lever1),  
    lever_threatened(Lever2),  
    nearest_zombie_distance_to_lever(Lever1, ZombieDistance1),  
    nearest_zombie_distance_to_lever(Lever2, ZombieDistance2),  
    ZombieDistance1 #> ZombieDistance2.
```

% Lever loses if another lever is logically better.

defeated_lever_target(Actor, Lever) :-

```
    lever_available(Actor, Lever),  
    better_lever(Actor, BetterLever, Lever),  
    BetterLever \= Lever.
```

preferred_lever(Actor, Lever) :-

```
    lever_available(Actor, Lever),  
    not defeated_lever_target(Actor, Lever).
```

BARRICADE STRENGTH STATUS

% Barricade is damaged below full reinforcement.

barricade_damaged(Barricade) :-

 barricade_reinforcement(Barricade, Reinforcement),
 Reinforcement #< 100.

% Barricade is in critical condition at 25 or below.

barricade_critical(Barricade) :-

 barricade_reinforcement(Barricade, Reinforcement),
 Reinforcement #=< 25.

% Barricade becomes urgent when a zombie is attacking it.

urgent_barricade(Actor, Barricade) :-

 barricade_visible(Actor, Barricade),
 zombie_at_barricade(_Zombie, Barricade).

BARRICADE PRECEDENCE COMPARISON

% If two barricades are urgent, prefer a critical one

% over one that is not yet critical.

better_barricade(Actor, Barricade1, Barricade2) :-

urgent_barricade(Actor, Barricade1),

urgent_barricade(Actor, Barricade2),

Barricade1 \= Barricade2,

barricade_critical(Barricade1),

not barricade_critical(Barricade2).

% If both urgent barricades are in the same condition,

% prefer the closer one.

better_barricade(Actor, Barricade1, Barricade2) :-

urgent_barricade(Actor, Barricade1),

urgent_barricade(Actor, Barricade2),

Barricade1 \= Barricade2,

closer(Actor, Barricade1, Barricade2).

defeated_barricade_target(Actor, Barricade) :-

urgent_barricade(Actor, Barricade),

better_barricade(Actor, BetterBarricade, Barricade),

BetterBarricade \= Barricade.

preferred_barricade(Actor, Barricade) :-

urgent_barricade(Actor, Barricade),

not defeated_barricade_target(Actor, Barricade).

FULL PROGRAM SAMPLE - *subject to change with team input and concerns

innocent(player1).

health(player1, 3).

levers_to_win(15).

levers_pulled_count(0).

lever_visible(player1, lever1).

lever_visible(player1, lever2).

lever_state(lever1, unpulled).

lever_state(lever2, unpulled).

distance(player1, lever1, 3).

distance(player1, lever2, 7).

nearest_zombie_distance_to_lever(lever1, 2).

nearest_zombie_distance_to_lever(lever2, 20).

barricade_visible(player1, window1).

barricade_reinforcement(window1, 80).

distance(player1, window1, 6).

zombie(zombie1).

distance(player1, zombie1, 8).

distance(zombie1, window1, 8).

can_act(Actor) :-

 health(Actor, Health),

 Health #> 0.

critical_health(Actor) :-

 health(Actor, Health),

 Health #=< 1.

low_health(Actor) :-

 health(Actor, Health),

 Health #=< 2.

dead(Actor) :-
 health(Actor, Health),
 Health #=< 0.

immediate_danger_from(Actor, Zombie) :-
 zombie(Zombie),
 distance(Actor, Zombie, Distance),
 Distance #=< 2.

immediate_danger(Actor) :-
 immediate_danger_from(Actor, _Zombie).

levers_remaining :-
 levers_to_win(Required),
 levers_pulled_count(Current),
 Current #< Required.

lever_available(Actor, Lever) :-
 lever_visible(Actor, Lever),
 lever_state(Lever, unpulled).

visible_available_lever(Actor) :-
 lever_available(Actor, _Lever).

at_lever(Actor, Lever) :-
 distance(Actor, Lever, Distance),
 Distance #=< 1.

lever_threatened(Lever) :-
 nearest_zombie_distance_to_lever(Lever, Distance),
 Distance #=< 5.

lever_safe(Lever) :-
 nearest_zombie_distance_to_lever(Lever, Distance),
 Distance #> 5.

closer(Actor, Target1, Target2) :-
 distance(Actor, Target1, Distance1),
 distance(Actor, Target2, Distance2),
 Distance1 #< Distance2.

better_lever(Actor, Lever1, Lever2) :-

```
lever_available(Actor, Lever1),  
lever_available(Actor, Lever2),  
Lever1 \= Lever2,  
lever_safe(Lever1),  
lever_threatened(Lever2).
```

```
better_lever(Actor, Lever1, Lever2) :-  
    lever_available(Actor, Lever1),  
    lever_available(Actor, Lever2),  
    Lever1 \= Lever2,  
    lever_safe(Lever1),  
    lever_safe(Lever2),  
    closer(Actor, Lever1, Lever2).
```

```
better_lever(Actor, Lever1, Lever2) :-  
    lever_available(Actor, Lever1),  
    lever_available(Actor, Lever2),  
    Lever1 \= Lever2,  
    lever_threatened(Lever1),  
    lever_threatened(Lever2),  
    nearest_zombie_distance_to_lever(Lever1, ZombieDistance1),  
    nearest_zombie_distance_to_lever(Lever2, ZombieDistance2),  
    ZombieDistance1 #> ZombieDistance2.
```

```
defeated_lever_target(Actor, Lever) :-  
    lever_available(Actor, Lever),  
    better_lever(Actor, BetterLever, Lever),  
    BetterLever \= Lever.
```

```
preferred_lever(Actor, Lever) :-  
    lever_available(Actor, Lever),  
    not defeated_lever_target(Actor, Lever).
```

```
at_barricade(Actor, Barricade) :-  
    distance(Actor, Barricade, Distance),  
    Distance #=< 1.
```

```
zombie_at_barricade(Zombie, Barricade) :-  
    zombie(Zombie),  
    distance(Zombie, Barricade, Distance),  
    Distance #=< 1.
```

```
urgent_barricade(Actor, Barricade) :-  
    barricade_visible(Actor, Barricade),  
    zombie_at_barricade(_Zombie, Barricade).
```

```
barricade_damaged(Barricade) :-  
    barricade_reinforcement(Barricade, Reinforcement),  
    Reinforcement #< 100.
```

```
barricade_critical(Barricade) :-  
    barricade_reinforcement(Barricade, Reinforcement),  
    Reinforcement #=< 25.
```

```
better_barricade(Actor, Barricade1, Barricade2) :-  
    urgent_barricade(Actor, Barricade1),  
    urgent_barricade(Actor, Barricade2),  
    Barricade1 \= Barricade2,  
    barricade_reinforcement(Barricade1, Reinforcement1),  
    barricade_reinforcement(Barricade2, Reinforcement2),  
    Reinforcement1 #< Reinforcement2.
```

```
better_barricade(Actor, Barricade1, Barricade2) :-  
    urgent_barricade(Actor, Barricade1),  
    urgent_barricade(Actor, Barricade2),  
    Barricade1 \= Barricade2,  
    barricade_reinforcement(Barricade1, Reinforcement1),  
    barricade_reinforcement(Barricade2, Reinforcement2),  
    Reinforcement1 #= Reinforcement2,  
    closer(Actor, Barricade1, Barricade2).
```

```
defeated_barricade_target(Actor, Barricade) :-  
    urgent_barricade(Actor, Barricade),  
    better_barricade(Actor, BetterBarricade, Barricade),  
    BetterBarricade \= Barricade.
```

```
preferred_barricade(Actor, Barricade) :-  
    urgent_barricade(Actor, Barricade),  
    not defeated_barricade_target(Actor, Barricade).
```

```
candidate_action(Actor, run_from(Entity)) :-  
    can_act(Actor),  
    immediate_danger_from(Actor, Entity).
```

```
candidate_action(Actor, hold_barricade(Barricade)) :-  
    can_act(Actor),  
    innocent(Actor),  
    preferred_barricade(Actor, Barricade),  
    at_barricade(Actor, Barricade).
```

```
candidate_action(Actor, move_to_barricade(Barricade)) :-  
    can_act(Actor),  
    innocent(Actor),  
    preferred_barricade(Actor, Barricade),  
    not at_barricade(Actor, Barricade).
```

```
candidate_action(Actor, pull_lever(Lever)) :-  
    can_act(Actor),  
    innocent(Actor),  
    preferred_lever(Actor, Lever),  
    at_lever(Actor, Lever).
```

```
candidate_action(Actor, move_to_lever(Lever)) :-  
    can_act(Actor),  
    innocent(Actor),  
    preferred_lever(Actor, Lever),  
    not at_lever(Actor, Lever).
```

```
candidate_action(Actor, hunt_lever) :-  
    can_act(Actor),  
    innocent(Actor),  
    levers_remaining,  
    not visible_available_lever(Actor).
```

```
candidate_action(Actor, wait) :-  
    can_act(Actor),  
    innocent(Actor),  
    not levers_remaining,  
    not urgent_barricade(Actor, _Barricade),  
    not immediate_danger(Actor).
```

defeated(Actor, hunt_lever) :-
 urgent_barricade(Actor, _Barricade).

defeated(Actor, move_to_lever(_Lever)) :-
 urgent_barricade(Actor, _Barricade).

defeated(Actor, pull_lever(_Lever)) :-
 urgent_barricade(Actor, _Barricade).

defeated(Actor, move_to_barricade(_Barricade)) :-
 immediate_danger(Actor).

defeated(Actor, hold_barricade(_Barricade)) :-
 immediate_danger(Actor).

defeated(Actor, hunt_lever) :-
 immediate_danger(Actor).

defeated(Actor, move_to_lever(_Lever)) :-
 immediate_danger(Actor).

defeated(Actor, pull_lever(_Lever)) :-
 immediate_danger(Actor).

action(Actor, Action) :-
 candidate_action(Actor, Action),
 not defeated(Actor, Action).